

Air Quality Intelligence System

Advanced Data Warehouse Technologies

Péter Dobosi (MW79ON)

University of Pannonia

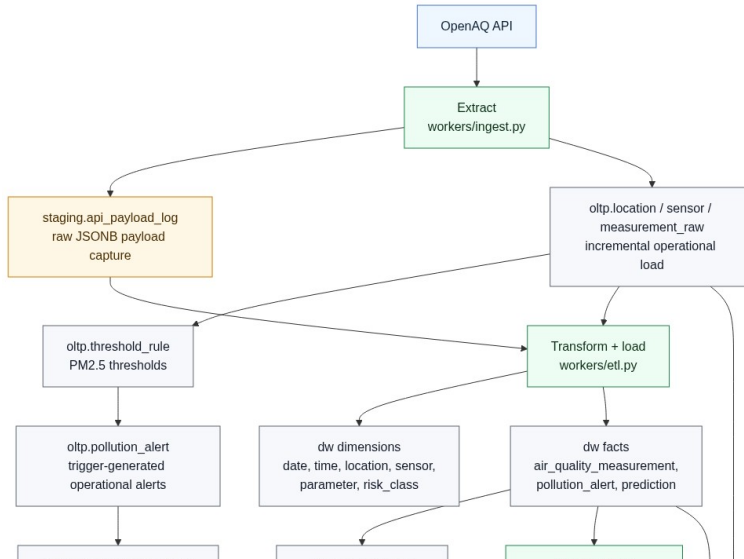
2025/26 Spring

Problem Statement

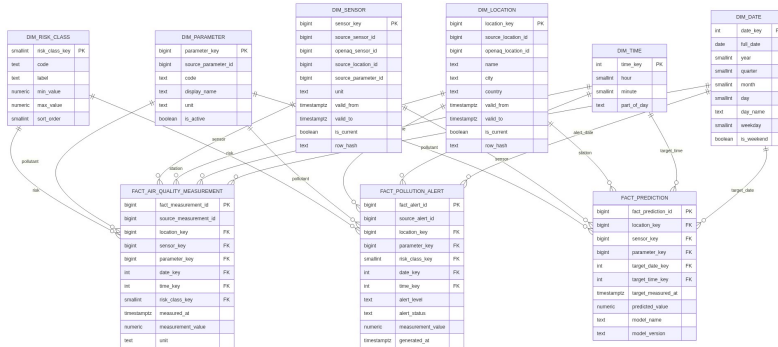
- Air quality data is voluminous and time-series in nature
- Raw API data is unstructured (JSON) and unsuitable for analytics
- Need: a well-modeled analytical warehouse with ETL, SCD, ML predictions, and dashboards

Goal: Build an end-to-end data warehouse pipeline from ingestion through prediction to interactive analytical dashboards.

Architecture & Data Flow



Star Schema Design (Fact Constellation)



Three fact tables sharing conformed dimensions (galaxy schema pattern).

- Python ETL script: extract from OLTP → transform → load into DW
- Watermark-based incremental extraction (only new rows)
- **SCD Type 2** on `dim_location` and `dim_sensor`:
 - Hash-based change detection (`row_hash`)
 - Expire old row (`valid_to`, `is_current = false`)
 - Insert new version (`valid_from = now()`)
- Conformed date/time dimensions pre-populated

Demo: Air Quality Overview Dashboard

- KPIs: latest value, weekly average, station count, open alerts
- Trend chart with daily aggregation (avoids spaghetti)
- Worst station metric + latest-by-station table
- Sidebar filters: city, station, pollutant, date range

Live demo: <https://mw79on-demo.online> → Air Quality Overview

Demo: High-Risk Periods

- Alert heatmap: weekday $\times$ hour concentration
- Severity mix bar chart
- Alerts by pollutant and by station (top 10)
- Recent high-risk alert table

Live demo: → [High-Risk Periods page](#)

Demo: Prediction Insights

- Random Forest model trained on DW historical data
- Features: hour, weekday, month, recent averages, location
- Chart: actual vs. predicted PM2.5 concentration
- Metrics: MAE $\approx$ 0.91, RMSE $\approx$ 1.21, $R^2 \approx$ 0.73
- Predictions stored as `fact_prediction` in the DW

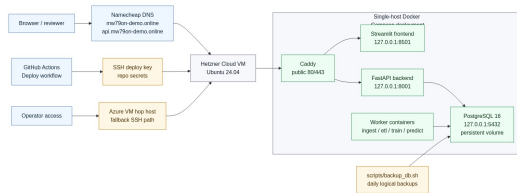
Live demo: → Prediction Insights page

Automated Refresh & Scheduling

- Orchestration script: `scripts/run_pipeline.sh`
- Modes: `full`, `ingest-etl`, `predict-only`, `train-predict`
- Stage gating: failure in ingest → ETL skipped
- Cron: ingest-etl every 3 hours, predict daily
- Logs written to `logs/` with status JSON

Deployment

- Hetzner Cloud VM
- Docker Compose (5 services)
- Caddy: automatic HTTPS
- GitHub Actions CI/CD
- Single `git push` → deployed



Summary & Lessons Learned

- Star schema + SCD2 enables rich historical analytics
- Incremental ETL keeps the warehouse fresh without full rebuilds
- ML predictions as first-class DW facts enable comparison dashboards
- Daily aggregation solves multi-station chart readability
- Automated pipelines + CI/CD = hands-off operation

Live system: <https://mw79on-demo.online>

Source code: <https://github.com/dobosipeter/advanced-data-warehouse-and-database-management-systems-submission>